

Open in app ↗

 Search WriteWelcome Offer Access to everything. Now 30% off. [Upgrade now](#)

Data Preprocessing: Encoding and Feature Scaling



Marishetty Ramyakrishna 9 min read · 9 hours ago



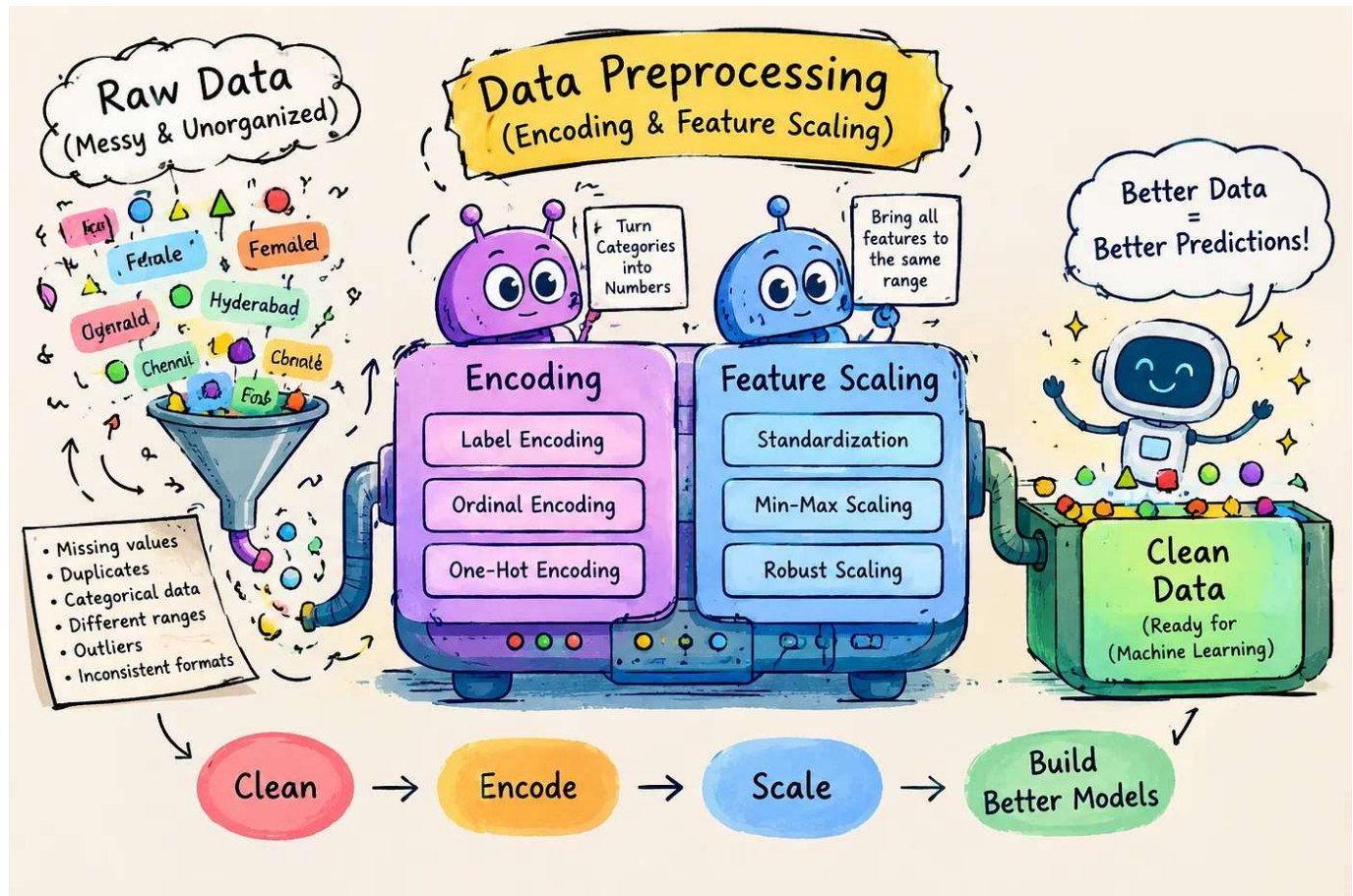
Machine Learning models work with numbers, but real-world datasets are rarely clean and ready to use.

A dataset may contain:

- Missing values
- Duplicate records
- Categorical data such as Male, Female, Hyderabad, Chennai
- Numerical features with very different ranges
- Outliers
- Inconsistent data formats

Before building a Machine Learning model, we need to prepare the data properly.

This process is called **Data Preprocessing**.



In this article, we will focus on two important preprocessing techniques:

1. Encoding
2. Feature Scaling

Let's understand them step by step with simple examples and Python code.

What is Data Preprocessing?

Data preprocessing is the process of cleaning and transforming raw data into a format that can be effectively used by Machine Learning algorithms.

For example, suppose we have the following dataset:

Age	Gender	Salary
22	Male	25,000
25	Female	30,000
30	Male	50,000
35	Female	75,000

Here:

- Age → Numerical
- Salary → Numerical
- Gender → Categorical

Machine Learning algorithms generally work with numerical values.

Therefore, Gender needs to be converted into numerical form.

Also, Age and Salary have very different ranges.

So, depending on the Machine Learning algorithm, we may need to scale them.

This is where **Encoding and Feature Scaling** become important.

Part 1: Encoding

What is Encoding?

Encoding is the process of converting categorical data into numerical values so that Machine Learning algorithms can understand it.

For example:

Gender

Male
Female
Male
Female

A Machine Learning algorithm cannot directly perform mathematical operations on these text values.

We can transform them into numbers:

Male → 0

Female → 1

This process is called **Encoding**.

Types of Encoding

Some commonly used encoding techniques are:

1. Label Encoding
2. Ordinal Encoding
3. One-Hot Encoding
4. Binary Encoding
5. Target Encoding

In this article, we will mainly focus on the most commonly used techniques:

- Label Encoding
- Ordinal Encoding
- One-Hot Encoding

Label Encoding

Label Encoding assigns a unique numerical value to each category.

For example:

Gender

Male
Female
Male
Female

After Label Encoding:

Male → 1
Female → 0

So the dataset becomes:

Gender	Encoded
Male	1
Female	0
Male	1
Female	0

Label Encoding using Python

First, let's create a small dataset.

```
import pandas as pd
```

```
data = {  
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male']  
}  
df = pd.DataFrame(data)  
print(df)
```

Output:

```
Gender  
0    Male  
1  Female  
2    Male  
3  Female  
4    Male
```

Now let's use LabelEncoder .

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()  
df['Gender_Encoded'] = encoder.fit_transform(df['Gender'])  
print(df)
```

Output:

	Gender	Gender_Encoded
0	Male	1
1	Female	0
2	Male	1
3	Female	0
4	Male	1

Important Point About Label Encoding

Label Encoding can create an **artificial order** between categories.

For example:

Red	→	0
Blue	→	1
Green	→	2

The model may interpret this as:

Green > Blue > Red

But colors do not actually have this numerical order.

Therefore, Label Encoding is generally more suitable for:

- Binary categories

- Ordinal categories where order actually exists

For example:

```
Low    → 0
Medium → 1
High   → 2
```

Here the order is meaningful.

Ordinal Encoding

Ordinal Encoding is used when categorical values have a meaningful order.

For example:

```
Education Level
```

```
School
Bachelor
Master
PhD
```

There is a natural order:

```
School < Bachelor < Master < PhD
```

We can encode them as:

```
School    → 0  
Bachelor  → 1  
Master    → 2  
PhD       → 3
```

Ordinal Encoding using Python

```
from sklearn.preprocessing import OrdinalEncoder
```

```
data = {  
    'Education': ['School', 'Bachelor', 'Master', 'PhD', 'Bachelor']  
}  
df = pd.DataFrame(data)  
encoder = OrdinalEncoder(  
    categories=[['School', 'Bachelor', 'Master', 'PhD']]  
)  
df['Education_Encoded'] = encoder.fit_transform(  
    df[['Education']]  
)  
print(df)
```

Output:

```
Education  Education_Encoded  
0      School                0.0  
1   Bachelor                1.0  
2      Master                2.0
```

3	PhD	3.0
4	Bachelor	1.0

One-Hot Encoding

One-Hot Encoding is one of the most commonly used encoding techniques.

It creates a separate column for each category.

Suppose we have:

City

Hyderabad
Chennai
Delhi
Hyderabad

One-Hot Encoding creates:

City	Chennai	Delhi	Hyderabad
Chennai	1	0	0
Delhi	0	1	0
Hyderabad	0	0	1

Each category gets its own column.

The value is:

```
1 → Category is present  
0 → Category is not present
```

One-Hot Encoding using Pandas

```
import pandas as pd
```

```
data = {  
    'City': ['Hyderabad', 'Chennai', 'Delhi', 'Hyderabad']  
}  
  
df = pd.DataFrame(data)  
  
encoded_df = pd.get_dummies(df, columns=['City'])  
  
print(encoded_df)
```

Output:

	City_Chennai	City_Delhi	City_Hyderabad
0	False	False	True
1	True	False	False
2	False	True	False
3	False	False	True

If you want integer values instead of Boolean values:

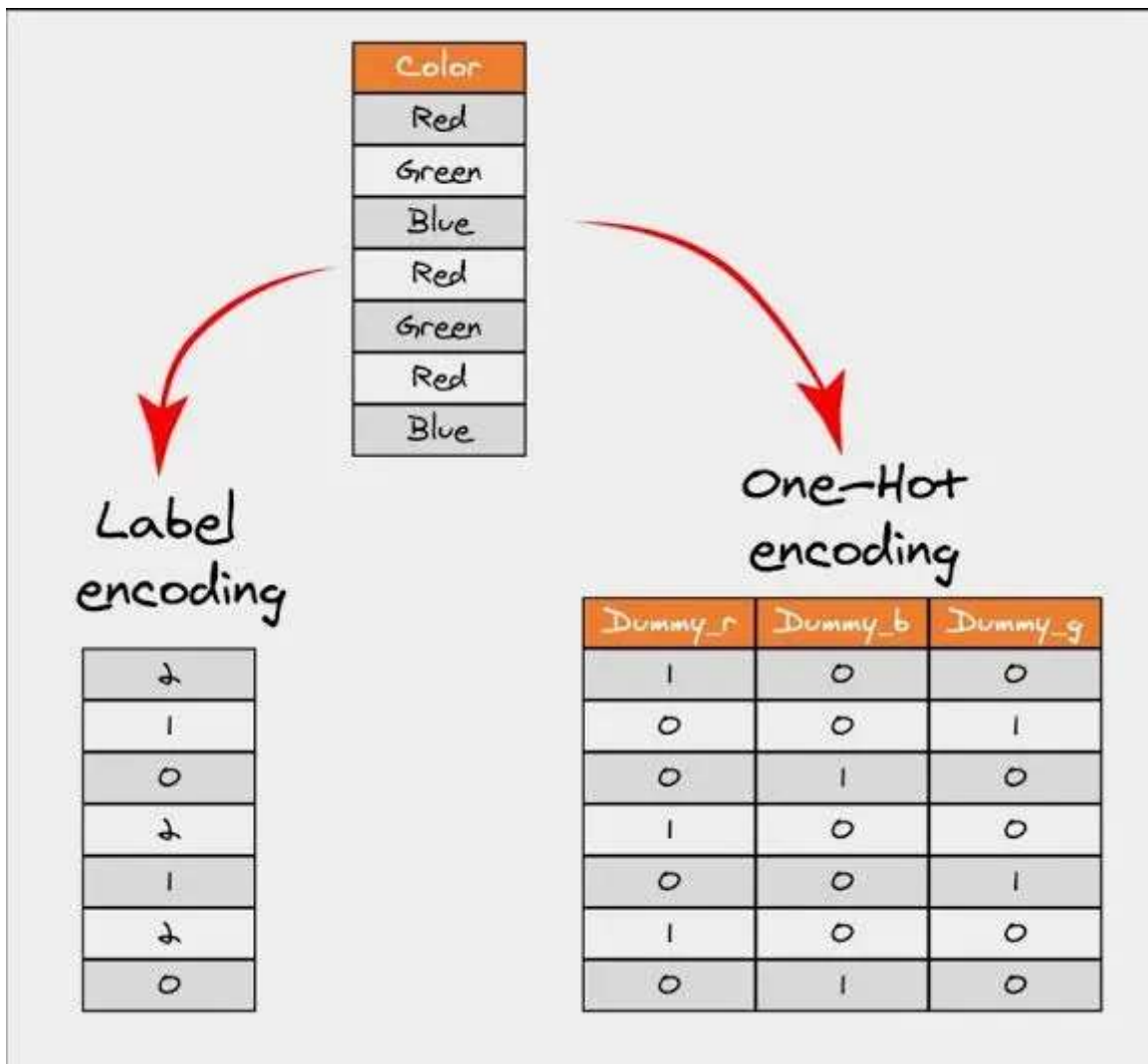
```
encoded_df = pd.get_dummies(  
    df,  
    columns=['City'],  
    dtype=int  
)
```

```
print(encoded_df)
```

Output:

	City_Chennai	City_Delhi	City_Hyderabad	
0	0	0		1
1	1	0		0
2	0	1		0
3	0	0		1

Label Encoding vs One-Hot Encoding

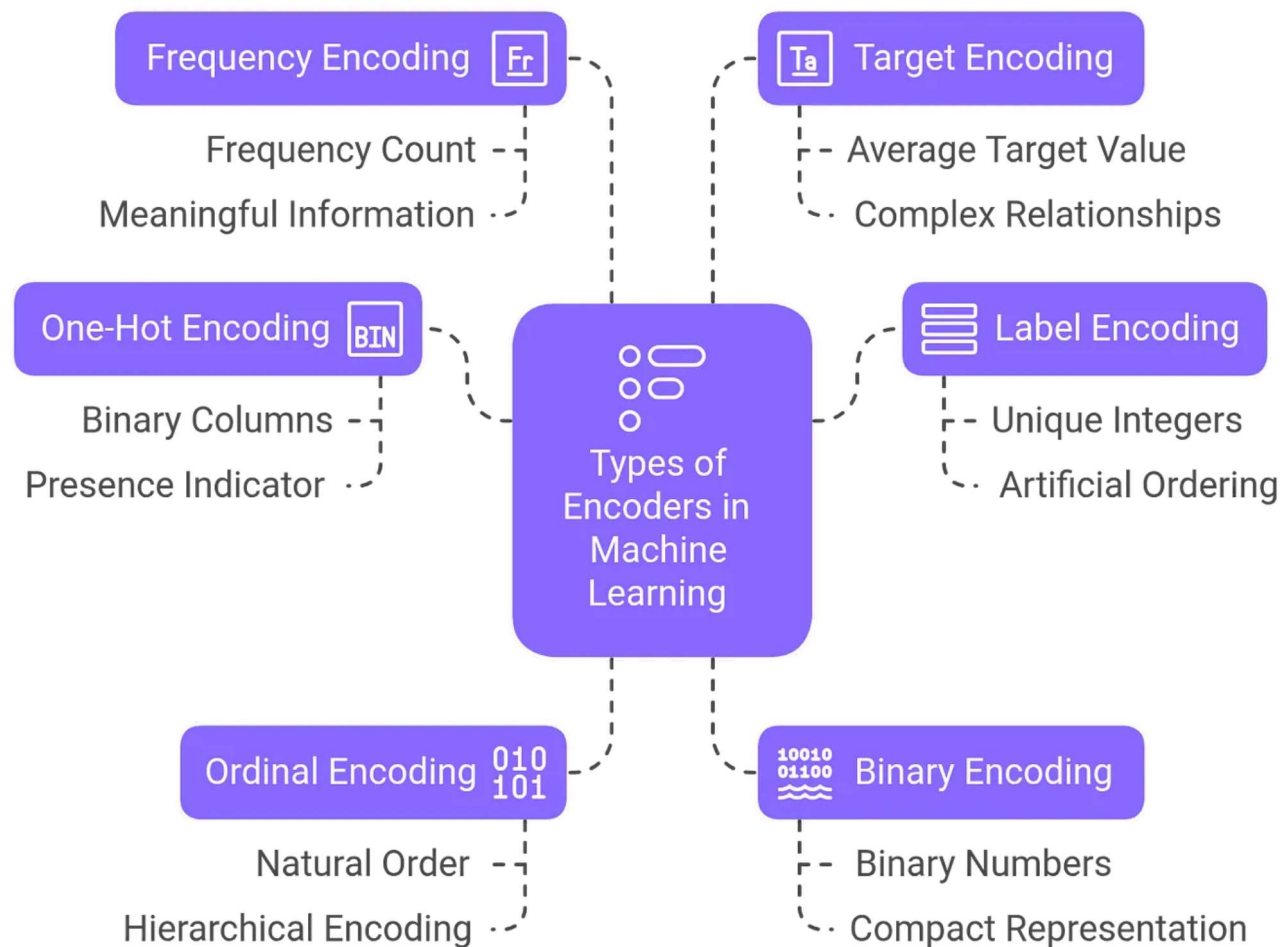


Label encoding vs One-Hot-encoding

Simple rule:

If the categories have **no natural order**, One-Hot Encoding is usually safer.

If the categories have a **meaningful order**, Ordinal Encoding can be appropriate.



Part 2: Feature Scaling

Now let's move to the second important preprocessing technique.

What is Feature Scaling?

Feature Scaling is the process of bringing numerical features to a similar scale.

Consider this dataset:

Age	Salary
22	25000
25	40000
30	60000
35	90000

Here:

Age → 22 to 35
Salary → 25,000 to 90,000

The ranges are very different.

Some Machine Learning algorithms can be affected by these differences in scale.

Feature Scaling helps bring numerical features to comparable ranges.

Why is Feature Scaling Needed?

Suppose we have:

```
Age = 30  
Salary = 60000
```

The numerical value of Salary is much larger than Age.

Some algorithms calculate distances or rely on optimization.

For example, distance-based algorithms include:

- K-Nearest Neighbors (KNN)
- K-Means Clustering
- Support Vector Machines (SVM)

Scaling can also be important for algorithms that use gradient-based optimization, such as:

- Logistic Regression
- Linear Regression
- Neural Networks

Without proper scaling, a feature with a larger numerical range can have an unnecessarily large influence on the calculations.

Common Feature Scaling Techniques

The most commonly used techniques are:

1. Standardization
2. Min-Max Scaling
3. Robust Scaling

Let's understand each one.

Standardization

Standardization transforms data so that it has approximately:

Mean = 0
Standard Deviation = 1

The formula is:

$$z = (x - \mu) / \sigma$$



Where:

- x = Original value
- μ = Mean
- σ = Standard deviation

- z = Standardized value

Simple Example

Suppose:

Mean = 50
Standard Deviation = 10
Value = 70

Using the formula:

$$z = (70 - 50) / 10$$

$$z = 20 / 10$$

$$z = 2$$

So the standardized value is:

2

Standardization using Python

We can use `StandardScaler` from Scikit-learn.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler

data = {
    'Age': [22, 25, 30, 35, 40],
    'Salary': [25000, 30000, 45000, 60000, 80000]
}

df = pd.DataFrame(data)
scaler = StandardScaler()
df_scaled = scaler.fit_transform(df)
print(df_scaled)
```

The resulting values are centered around zero.

Min-Max Scaling

Min-Max Scaling transforms values into a fixed range, usually:

0 to 1

The formula is:

$$X_{\text{scaled}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Where:

- x = Original value
- $x_{\min}$ = Minimum value
- $x_{\max}$ = Maximum value

Simple Example

Suppose:

```
Minimum = 10  
Maximum = 50  
Value = 30
```

Then:

```
 $x_{\text{scaled}} = (30 - 10) / (50 - 10)$ 
```

```
 $x_{\text{scaled}} = 20 / 40$ 
```

```
 $x_{\text{scaled}} = 0.5$ 
```

So:

30 → 0.5

Min-Max Scaling using Python

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
```

```
data = {
    'Age': [22, 25, 30, 35, 40],
    'Salary': [25000, 30000, 45000, 60000, 80000]
}
df = pd.DataFrame(data)
scaler = MinMaxScaler()
df_scaled = scaler.fit_transform(df)
print(df_scaled)
```

Now the values are transformed into a range between 0 and 1.

Robust Scaling

Robust Scaling is useful when the dataset contains **outliers**.

It uses the:

- Median
- Interquartile Range (IQR)

The formula is:

$$X_{\text{scaled}} = (X - \text{Median}) / \text{IQR}$$

Where:

$$\text{IQR} = Q3 - Q1$$

Because Robust Scaling uses the median and IQR instead of the mean and standard deviation, it is less affected by extreme values.

Robust Scaling using Python

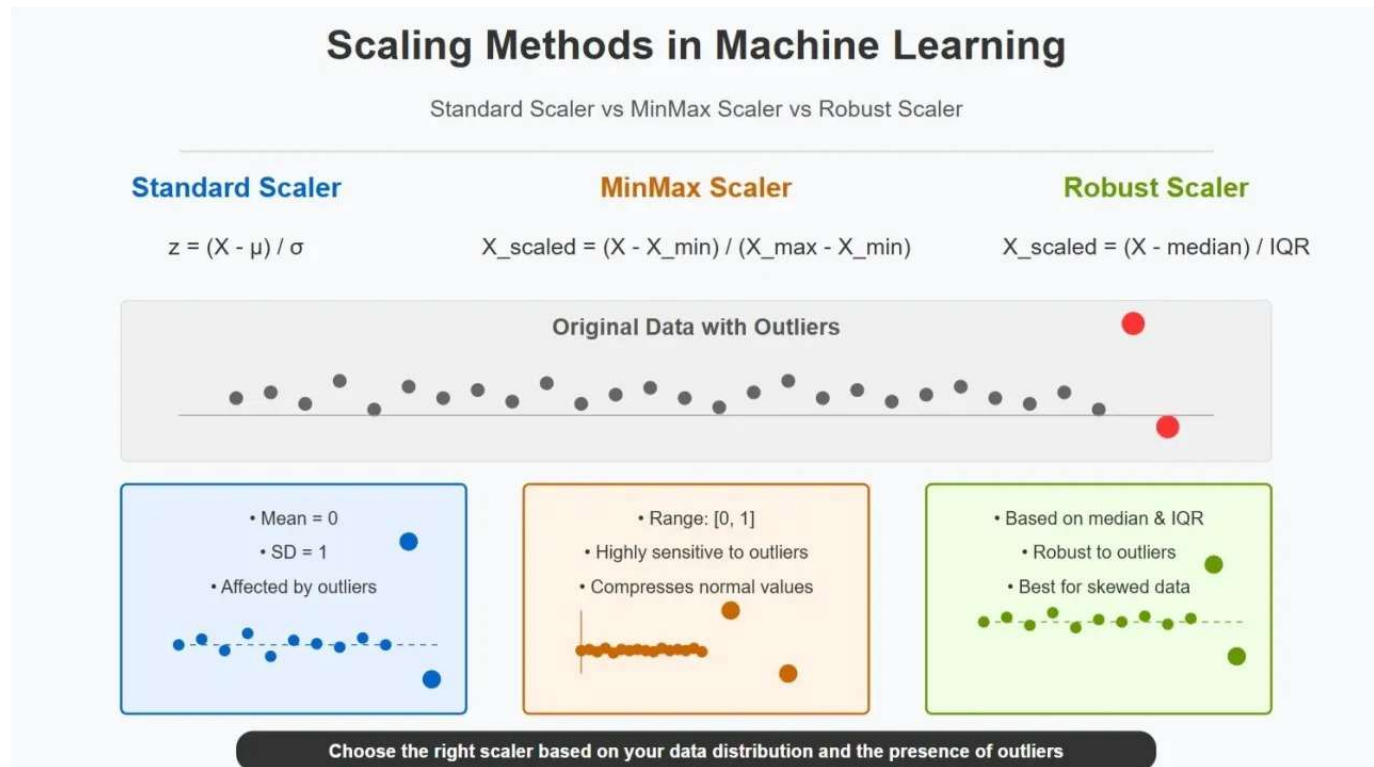
```
import pandas as pd
from sklearn.preprocessing import RobustScaler
```

```
data = {
    'Salary': [25000, 30000, 35000, 40000, 500000]
}
df = pd.DataFrame(data)
scaler = RobustScaler()
df['Salary_Scaled'] = scaler.fit_transform(
    df[['Salary']]
)
print(df)
```

Here, 500000 is much larger than the other values.

Robust Scaling can be useful when such extreme values are present.

Standardization vs Min-Max Scaling vs Robust Scaling



Feature scaling is generally applied to **numerical features**.

Categorical columns should first be encoded appropriately.

For example:

Age	→ Scaling
Salary	→ Scaling
Gender	→ Encoding
City	→ Encoding

So the preprocessing process could look like:

```
Raw Dataset
  ↓
Data Cleaning
  ↓
Handle Missing Values
  ↓
Encode Categorical Features
  ↓
Scale Numerical Features
  ↓
Train Machine Learning Model
```

Very Important: Data Leakage

One of the most important things to remember during preprocessing is **data leakage**.

Suppose we divide our dataset into:

```
Training Data
Testing Data
```

We should **fit** the scaler only on the training data.

Correct approach:

```
scaler.fit(X_train)
```

```
X_train_scaled = scaler.transform(X_train)  
X_test_scaled = scaler.transform(X_test)
```

Or simply:

```
X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)
```

Why?

Because the test data should represent **unseen data**.

If we calculate scaling parameters using both training and test data, information from the test set can leak into the training process.

This can result in misleadingly good model performance.

fit(), transform(), and fit_transform()

These three methods are extremely important in Machine Learning preprocessing.

fit()

`fit()` learns the required parameters from the data.

For example, `StandardScaler` learns:

```
Mean  
Standard Deviation
```

`transform()`

`transform()` applies the parameters learned during `fit()`.

`fit_transform()`

`fit_transform()` performs both operations:

```
fit()  
+  
transform()
```

For training data, we commonly use:

```
X_train_scaled = scaler.fit_transform(X_train)
```

For testing data:

```
X_test_scaled = scaler.transform(X_test)
```

Remember:

```
Training Data → fit_transform()  
Testing Data → transform()
```

This is a very important Machine Learning interview concept.

Complete Example: Encoding + Scaling

Let's combine everything.

Suppose we have:

```
import pandas as pd
```

```
data = {  
    'Age': [22, 25, 30, 35, 40],  
    'Salary': [25000, 30000, 45000, 60000, 80000],  
    'Gender': ['Male', 'Female', 'Male', 'Female', 'Male']  
}  
  
df = pd.DataFrame(data)  
  
print(df)
```

Our dataset contains:

```
Age      → Numerical  
Salary   → Numerical  
Gender    → Categorical
```

Step 1: Encode the Categorical Feature

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()  
df['Gender'] = encoder.fit_transform(df['Gender'])  
print(df)
```

Step 2: Select Numerical Features

```
features = ['Age', 'Salary']
```

Step 3: Apply Standardization

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()  
df[features] = scaler.fit_transform(df[features])
```

```
print(df)
```

Now:

```
Gender → Encoded  
Age    → Scaled  
Salary → Scaled
```

The data is now much more suitable for Machine Learning algorithms that benefit from scaled numerical features.

Real-World Preprocessing Pipeline

In a real Machine Learning project, preprocessing can involve several steps.

A typical workflow is:

```
Raw Data  
  ↓  
Data Understanding  
  ↓  
Data Cleaning  
  ↓  
Handle Missing Values  
  ↓  
Remove Duplicates  
  ↓  
Handle Outliers  
  ↓  
Encode Categorical Variables  
  ↓  
Scale Numerical Variables  
  ↓  
Feature Selection
```

↓
Train-Test Split
↓
Machine Learning Model
↓
Model Evaluation

However, the exact order can vary depending on the dataset and the preprocessing strategy.

When Should You Use Scaling?

Scaling is especially important for algorithms based on:

Distance

- KNN
- K-Means
- DBSCAN

Margin / Distance

- SVM

Gradient-Based Optimization

- Logistic Regression
- Linear Regression
- Neural Networks

Tree-based algorithms such as:

- Decision Tree
- Random Forest
- XGBoost

generally do **not require feature scaling** because their splits are based on feature thresholds rather than distances between scaled feature values.

Encoding and Scaling: Simple Real-World Example

Imagine we are building a model to predict whether a customer will take a loan.

Our dataset contains:

Age	Salary	Gender	Education
25	30000	Male	Bachelor
30	50000	Female	Master
35	70000	Male	PhD

We can preprocess it like this:

Age
↓

Scaling

Salary

↓

Scaling

Gender

↓

Encoding

Education

↓

Ordinal Encoding

After preprocessing, all the information is represented in a numerical format suitable for a Machine Learning model.

Final Takeaway

Data preprocessing is one of the most important steps in a Machine Learning workflow.

Encoding helps us convert categorical information into numerical form.

Feature Scaling helps bring numerical features to comparable scales when the chosen algorithm benefits from it.

The most important concepts to remember are:

Categorical Data

↓

Encoding

↓

Numerical Data

↓

Feature Scaling



Machine Learning Model

And remember this important rule:

Training Data → `fit_transform()`

Testing Data → `transform()`

Understanding **what to use, why to use it, and when to use it** is much more important than simply memorizing the Python syntax.

Once you understand Encoding and Feature Scaling, you have a strong foundation for the Machine Learning preprocessing and model building.



Written by Marishetty Ramyakrishna

0 followers · 2 following

Follow

